Python 金融数据分析

第 8 章　SciPy与金融数值计算

主讲人:　王伟冠, 吕文俊
院　系:　经济学院金融系

上海大学

2026 年 10 月 14 日



1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

已有 NumPy，为什么还需要 SciPy?

- NumPy：保存数组、逐元素运算、统计汇总和随机抽样。
- SciPy：在数组与函数之上，提供概率计算、求根、插值、优化和积分算法。
- 学习顺序：概率与分位点 → 方程求根 → 拟合与插值 → 优化 → 积分。
- 每种工具都要说清输入、输出、单位，并检查计算结果。

与后续章节的分工

本章学习工具；第 9 章用于模拟与期权定价，第 10 章深入统计检验与投资组合优化。SymPy 是独立的符号计算库，放在章末选学。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

假设测量误差服从均值为 0、标准差为 1 的正态分布。误差不超过 1 的概率是多少？

```
from scipy.stats import norm
print(round(norm.pdf(0), 4))
print(round(norm.cdf(1), 4))
print(round(norm.cdf(1) - norm.cdf(-1), 4))
```

运行输出

```
0.3989
0.8413
0.6827
```

pdf 给出密度高度，不是单点概率；cdf(x) 给出不超过 x 的累计概率。最后一行表示误差位于 -1 到 1 的概率，约为 68.27%。

若希望 95% 的误差不超过某个阈值，应选择多大的阈值?

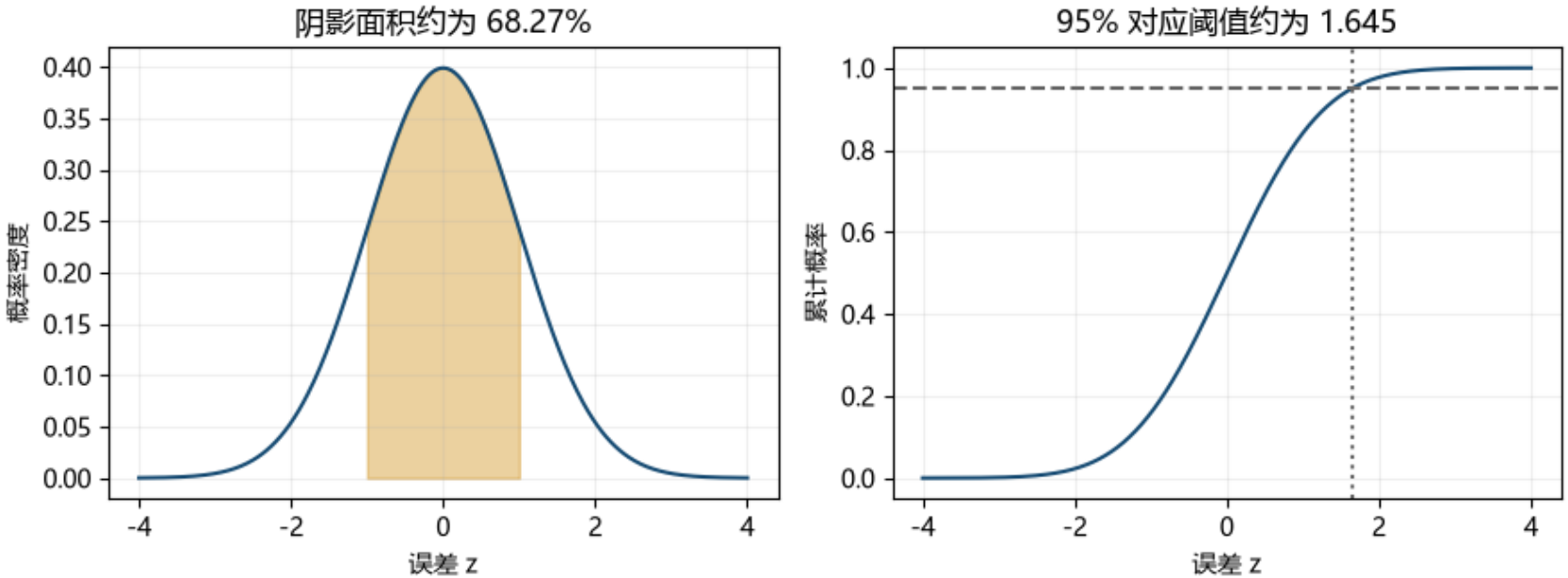
```
threshold = norm.ppf(0.95)
print(round(threshold, 4))
print(round(norm.cdf(threshold), 4))
print(np.round(norm.ppf([0.025, 0.975]), 4))
```

运行输出

```
1.6449
0.95
[-1.96  1.96]
```

ppf 是累计分布函数的反向查询。单侧 95% 阈值约为 1.645；中央 95% 区间约为 [-1.96, 1.96]，两者不能混淆。

看图：密度高度、区间面积与累计概率



左图阴影是区间概率；右图从累计概率 0.95 反查阈值。图形基于标准正态分布的理论计算。

假设日简单收益率服从均值 0.04%、标准差 1.5% 的正态分布；收益率用小数表示。

```
mu, sigma = 0.0004, 0.015
down_prob = norm.cdf(-0.03, loc=mu, scale=sigma)
lower_return = norm.ppf(0.05, loc=mu, scale=sigma)
print(f"收益低于-3%的概率：{down_prob:.2%}")
print(f"收益的5%分位点：{lower_return:.2%}")
```

运行输出

```
收益低于-3%的概率：2.13%
收益的5%分位点：-2.43%
```

loc 是均值，scale 是标准差。这里计算理论概率，而不是生成随机样本；正态假设仅用于教学，不代表真实市场尾部风险。

标准正态误差位于 -2 到 2 的概率是多少？中央 90% 区间的两个端点是多少？

先预测，再计算

参考答案见 Notebook 的折叠单元格。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. **数值求根：从方程到收益率**
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

求 $x^2=2$ 的正根。先写成 $h(x)=x^2-2=0$，再找函数值异号的区间。

```python
from scipy.optimize import root_scalar
def h(x):
    return x ** 2 - 2
print(h(1), h(2))
root_result = root_scalar(h, bracket=[1, 2], method="brentq")
print(round(root_result.root, 6), root_result.converged)
print(f"代回残差: {h(root_result.root):.2e}")
```

运行输出

```
-1 2
1.414214 True
代回残差: 1.17e-13
```

bracket 是根所在的区间。此方法要求函数在区间内连续、端点值异号；root 是数值解，converged 是收敛标志，残差检验代回后是否接近零。

自编例子：面值 100 元、每年付息 5 元，3 年后还本；恰在付息后估值。价格等于未来现金流的折现和。

```
years = np.array([1, 2, 3])
cashflows = np.array([5., 5., 105.])
def bond_price(y):
    return np.sum(cashflows / (1 + y) ** years)
print(round(bond_price(0.05), 4))
print(round(bond_price(0.08), 4))
```

运行输出

```
100.0
92.2687
```

y 是按年复利的到期收益率；每个元素对应一年末的现金流，单位为元。收益率 5% 时价格为 100 元，收益率升高时价格下降。

由价格反求到期收益率

同一债券市场价格为 98 元，求使模型价格等于 98 的年收益率。

```
market_price = 98.0
def price_gap(y):
    return bond_price(y) - market_price
ytm = root_scalar(price_gap, bracket=[0, 0.2], method="brentq")
print(f"年收益率: {ytm.root:.4%}")
print("收敛: ", ytm.converged)
print(f"价格残差: {price_gap(ytm.root):.2e} 元")
```

运行输出

```
年收益率: 5.7447%
收敛:  True
价格残差: -4.95e-11 元
```

先判断 0% 与 20% 对应价格是否跨过市场价。98 元低于面值，所以收益率应高于票息率 5%；还需核对现金流、复利频率和计价时点。

练习：价格变化后收益率怎样变化？

将同一债券价格改成 102 元。先判断收益率相对 5% 的大小，再求解并检查残差。

先预测，再计算

参考答案见 Notebook 的折叠单元格。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. **函数逼近：回归**
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

- 教材用一个"正弦 + 线性"函数用于后面的拟合、插值与积分：

$$f(x) = \sin(x) + 0.5\,x$$

- 它既有弯曲部分（考验拟合能力）又有趋势项，很适合演示逼近方法。
- 第 6 章我们用 `np.polyfit` 估计过 beta——那就是回归的金融应用；本章把它推广为通用的函数逼近工具。

零基础提醒

数学符号不必害怕：本章重点是"会调用、会读图、会解释"，推导细节留给专门课程。

```python
import numpy as np

def f(x):
    return np.sin(x) + 0.5 * x

x = np.linspace(-2 * np.pi, 2 * np.pi, 50)
reg = np.polyfit(x, f(x), deg=7)    # 7次多项式拟合
ry  = np.polyval(reg, x)            # 用拟合结果求值
np.mean((f(x) - ry) ** 2)           # 均方误差 MSE 约 0.0018
```

- `np.polyfit(x, y, deg)`：找一条 `deg` 次多项式，使它与数据点的误差平方和最小（最小二乘）。
- `np.polyval(reg, x)`：把拟合系数代入求值。
- MSE 越小拟合越好：1 次 ≈ 0.42，5 次 ≈ 0.054，7 次 ≈ 0.0018。

次数越高越好吗?

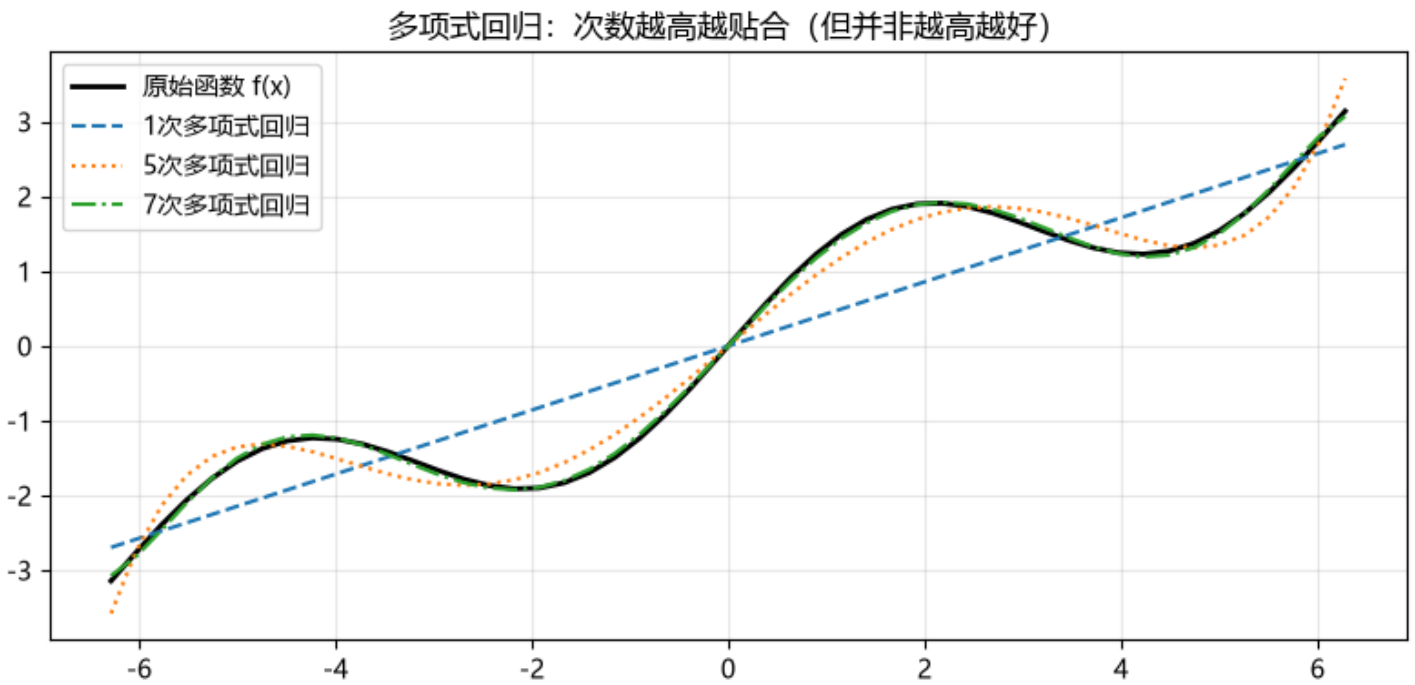


图 1: 1 次（直线）完全忽略波动；7 次几乎完美贴合

回归的两个优点

- **抗噪声**：数据带噪声时，回归曲线反而比数据点更接近真实函数——它在"平均掉"噪声。
- **不要求排序**：数据点顺序打乱也不影响结果。

过拟合警告

多项式次数过高时，曲线会穿过每个噪声点、在点之间剧烈震荡——对样本"背得滚瓜烂熟"，对新数据一塌糊涂。次数选择要配合样本外检验（后面的机器学习思想）。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. **函数逼近：插值**
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

样条插值：平滑地穿过每个点

```python
import scipy.interpolate as spi

xa = np.linspace(-2 * np.pi, 2 * np.pi, 12)  # 12个观测点
ya = f(xa)
ipo = spi.CubicSpline(xa, ya, extrapolate=False)    # 构造三次样条
xs = np.linspace(-2 * np.pi, 2 * np.pi, 200)
iy = ipo(xs)          # 在更密的网格上求值
```

- CubicSpline（构造）+ 对象调用（求值），与 polyfit+polyval 的分工一模一样。
- 三次样条——分段三次多项式，连接处一、二阶导数连续。

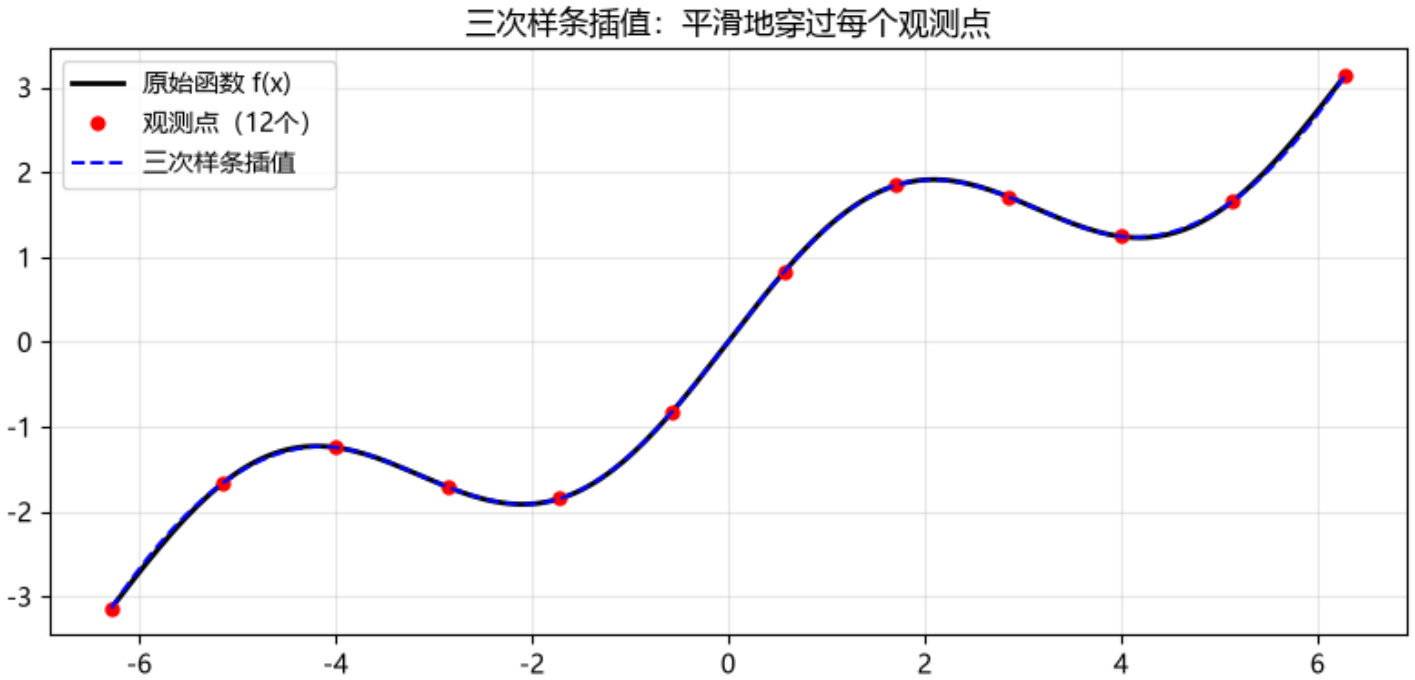


图 2: 只用 12 个点，三次样条就能平滑还原整条曲线

- 市场上只有整期限的国债利率：1 年、3 年、5 年、10 年...
- 定价需要 7.5 年的利率怎么办？——插值。

```
maturity = [1, 3, 5, 10]                # 已知期限（年）
rate     = [1.5, 2.1, 2.6, 3.2]         # 对应利率（%）
ipo = spi.CubicSpline(maturity, rate, extrapolate=False)
print(round(float(ipo(7.5)), 3))     # 单位：%
```

插值的限制

本例要求期限严格递增、输入有限；插值会通过节点，也会保留节点中的噪声。禁止区间外外推；样条不保证单调或无套利。利率是自编教学数据。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. **优化：寻找最优解**
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

求 $(x-3)^2+2$ 的最小值。手算知道最低点在 $x=3$，再用算法核对。

```
from scipy.optimize import minimize_scalar
def bowl(x):
    return (x - 3) ** 2 + 2
minimum = minimize_scalar(bowl, bounds=(0, 6), method="bounded")
print(round(minimum.x, 4), round(minimum.fun, 4))
print(minimum.success)
```

运行输出

```
3.0 2.0
True
```

x 是最优输入，fun 是目标函数值；求根找函数等于零的位置，优化找函数最小的位置。本例最小值为 2，根并不存在。

- 目标函数有多个"坑"（局部极小值），全局最小值藏在其中：

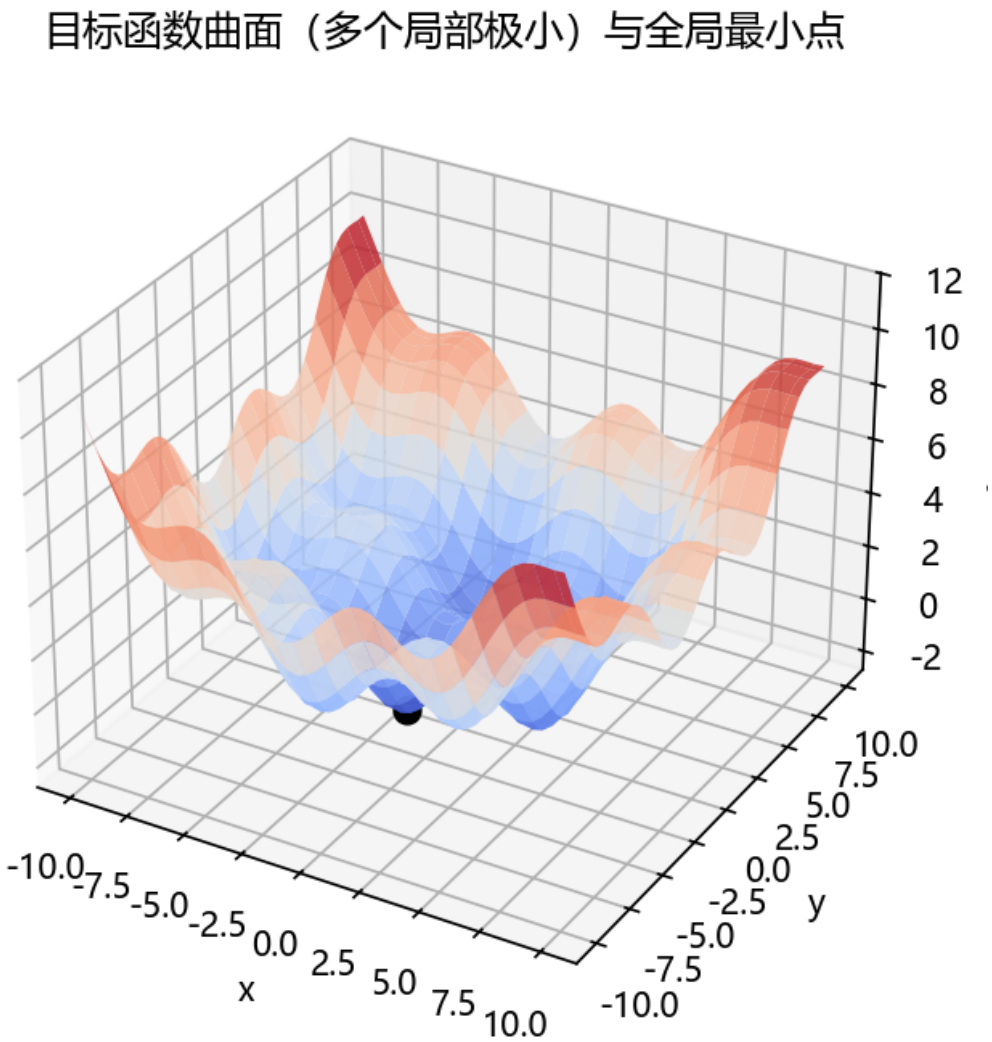


图 3: $f(x, y) = \sin x + 0.05x^2 + \sin y + 0.05y^2$，黑点为全局最小

```python
import scipy.optimize as sco

def fm(p):
    x, y = p
    return np.sin(x) + 0.05 * x ** 2 \
         + np.sin(y) + 0.05 * y ** 2

# 1. 在给定区间的网格上粗搜
opt1 = sco.brute(fm, ((-10, 10.1, 0.1), (-10, 10.1, 0.1)),
                 finish=None)          # 找到约 (-1.4, -1.4)
# 2. 局部：从粗搜结果出发精细下降
opt2 = sco.fmin(fm, opt1, xtol=0.001, ftol=0.001, disp=False)
opt2.round(4)    # [-1.4274, -1.4279], f = -1.7757
```

局部优化为什么受初值影响?

- 对多峰目标，不同初值可能使局部算法找到不同的局部极小值。
- 网格粗搜可以提供较好的初值，但有限网格不能保证找到连续问题的全局最优。
- 先判断问题结构：凸问题通常不需要先做全局搜索；多峰问题可比较多个初值或使用全局方法。
- 收敛标志只说明算法达到停止条件，不等于已经证明全局最优。

回到案例

比较从网格结果与从 (2, 2) 出发的目标值，再解释差异。Notebook 保留两次运行的实际结果。

- 预算 100 元买两种证券（单价均 10 元）；一年后状态 u/d 等概率，收益向量分别为 $(15, 5)$ 与 $(5, 12)$；效用 $u(w) = \sqrt{w}$。求最大化期望效用的购买数量。

```python
import math
def eu(p):       # 最小化"负效用" = 最大化效用
    s, b = p
    return -(0.5 * math.sqrt(s * 15 + b * 5)
           + 0.5 * math.sqrt(s * 5 + b * 12))
cons = [{"type": "ineq", "fun":
         lambda p: 100 - p[0] * 10 - p[1] * 10}]  # 预算约束
res = sco.minimize(eu, (5, 5), method="SLSQP",
                   bounds=((0, 10), (0, 10)),
                   constraints=cons)
print(res.success, res.x.round(2))
print("预算余量: ", round(100 - 10 * res.x.sum(), 6))
```

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. **积分：从面积到期权价格**
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

计算 0 到 1 上 x^2 的积分，手算为 1/3。算法返回的第二个数是什么？

```python
from scipy.integrate import quad
def square(x):
    return x ** 2
area, error_est = quad(square, 0, 1)
print(round(area, 8))
print(f"误差估计: {error_est:.2e}")
```

运行输出

```
0.33333333
误差估计: 3.70e-15
```

quad 返回积分值与绝对误差估计；误差估计不是精确误差，也不是数据或模型的不确定性。手算值与数值结果应在合理精度内一致。

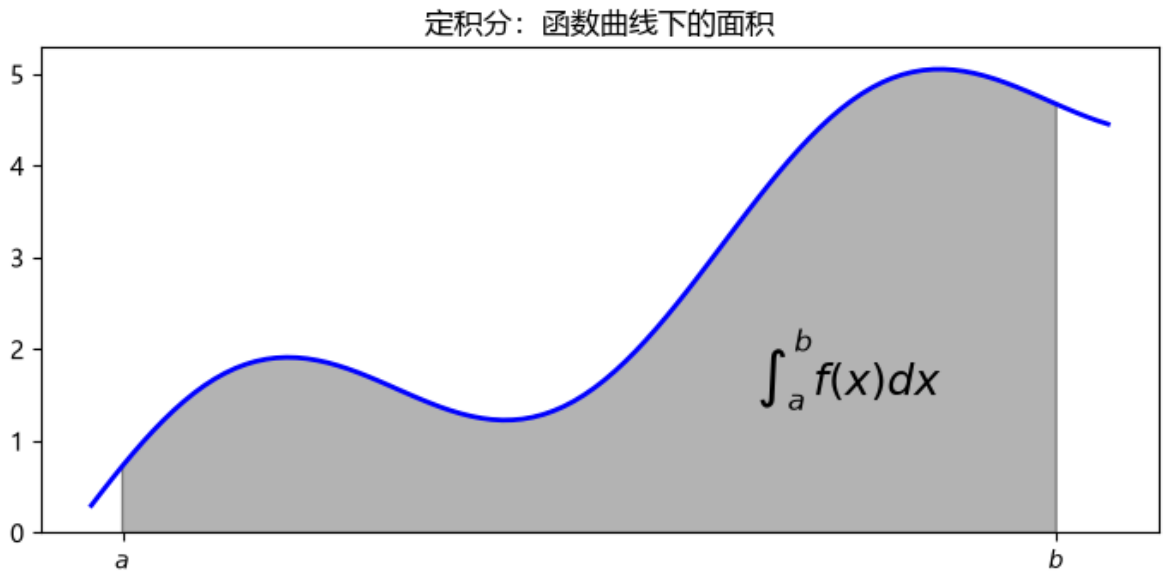


图 4: $\int_{0.5}^{9.5} f(x)\,dx$：衍生品定价的数学本质就是这类计算

数值积分：sci.quad

```python
import scipy.integrate as sci

a, b = 0.5, 9.5
value, error_est = sci.quad(f, a, b)
print(round(value, 6), f"{error_est:.2e}")
```

- quad 返回 (结果, 误差估计)，两者都应检查。
- 数据点形式则用 `sci.trapezoid`（梯形法则）。
- **金融意义**：期权在风险中性测度下的价值 = 期望折现收益；连续情形下，期望就是一个积分。

```python
rng = np.random.default_rng(1000)
N = 500000
x = rng.random(N) * (b - a) + a     # 区间内均匀撒点
np.mean(f(x)) * (b - a)             # 平均函数值 × 区间长度
# 结果 约 24.36，与 quad 基本一致
```

- 原理：$\int_a^b f(x)dx = (b-a) \cdot E[f(x)]$，用样本均值逼近期望。
- 样本越多越准，但收敛不单调，误差按 $1/\sqrt{N}$ 下降。

为什么金融偏爱蒙特卡洛?

高维（多资产、多期）积分数值方法会"维度爆炸"，蒙特卡洛的收敛速度却与维度无关——它是复杂衍生品定价的主力工具（下一章的主题）。

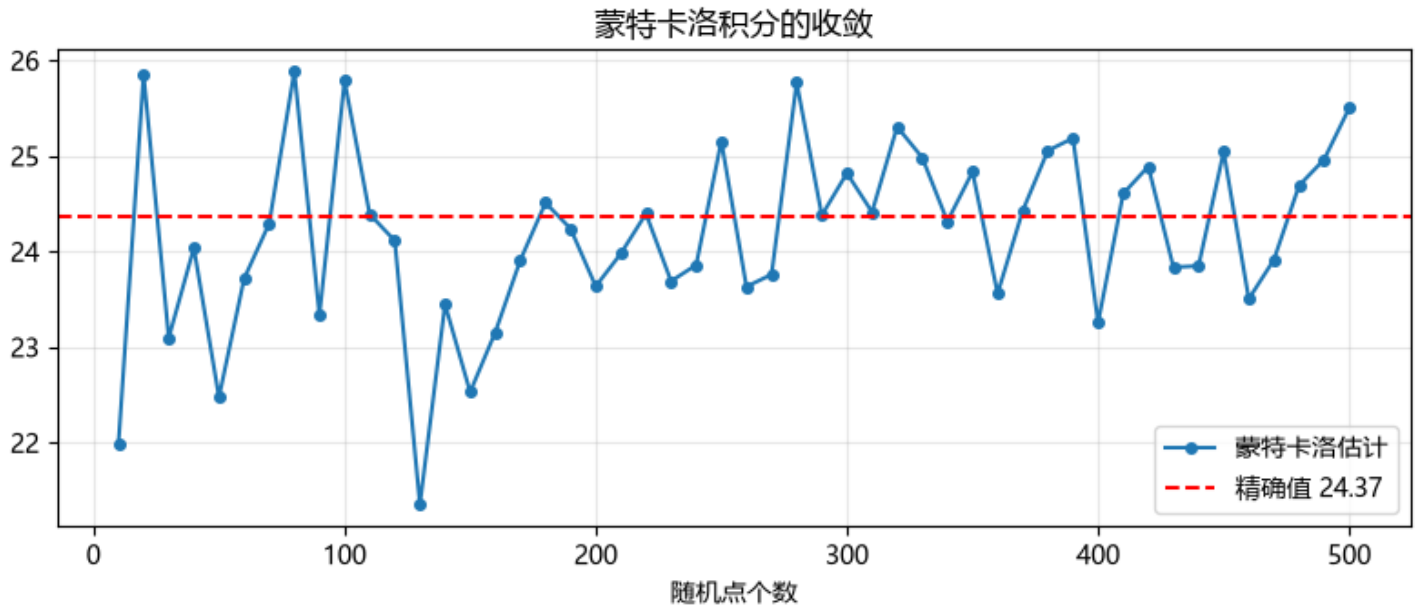


图 5: 估计值围绕精确值（红色虚线）震荡收窄

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. **结果检查**
9. 选学：符号计算 SymPy
10. 小结与练习

回到债券求根例子：收敛并不自动说明模型输入正确。用代回与方向判断共同检查。

```
print(ytm.converged)
print(round(bond_price(ytm.root), 6))
print(ytm.root > 0.05)
print(np.isclose(bond_price(ytm.root), 98, rtol=0, atol=1e-6))
```

运行输出

```
True
98.0
True
True
```

求根看收敛与残差；优化看 success、目标值和约束；积分看误差估计；插值先核对节点，再检查区间内形状。容差应符合单位与用途。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. **选学：符号计算 SymPy**
10. 小结与练习

符号对象：让 x 保持为 x

```python
import sympy as sy

x = sy.Symbol("x")          # x 是符号，不是数字
f_sym = sy.sin(x) + 0.5 * x    # 符号表达式
sy.solve(x ** 2 - 1, x)    # 解方程：[-1, 1]
sy.simplify((x ** 2 - 1) / (x - 1))    # 化简：x + 1
```

- NumPy 算的是"数"，SymPy 算的是"式子"。
- 在 Notebook 中，SymPy 表达式会以漂亮的数学公式显示。

```
sy.diff(f_sym, x)                          # 求导：cos(x) + 0.5
sy.integrate(f_sym, x)                     # 不定积分：0.25x^2 - cos(x)
sy.integrate(f_sym, (x, 0.5, 9.5)).evalf()   # 定积分：24.3748
```

- 与数值积分 quad 的结果在数值精度内一致——互相验证的好例子。
- 金融应用：期权价格对参数的导数（Delta、Gamma 等"希腊字母"）可用符号求导得到解析式。

数值 vs 符号

要"精确公式"用 SymPy；要"大规模快速计算"用 NumPy/SciPy。两者互补。

1. 从数组运算到数值算法
2. 概率分布：从抽样到概率
3. 数值求根：从方程到收益率
4. 函数逼近：回归
5. 函数逼近：插值
6. 优化：寻找最优解
7. 积分：从面积到期权价格
8. 结果检查
9. 选学：符号计算 SymPy
10. 小结与练习

本章小结：按问题选择工具

- 概率与阈值：`stats.norm.pdf/cdf/ppf`；密度高度不是概率。
- 方程求根：`root_scalar`；看连续性、区间、收敛与残差。
- 拟合与插值：`polyfit` 折中误差，`CubicSpline` 通过节点。
- 优化：`minimize_scalar/minimize`；目标值、初值、约束共同检查。
- 积分：`quad` 返回数值与误差估计；蒙特卡洛用抽样均值近似积分。
- 选学 SymPy：进行符号运算，注意它与 SciPy 的区别。



课后任务

1. 对 $g(x)=\cos(x)+0.2x$ 分别做 3 次与 9 次多项式回归，比较 MSE 并绘图。
2. 给定期限 $[1,2,5,10]$ 年、利率 $[1.6,1.9,2.5,3.1]$，用三次样条插值估计 3.5 年期利率。
3. 选做：把蒙特卡洛积分的样本数从 10^3 增加到 10^6，记录误差变化，验证 $1/\sqrt{N}$ 规律。

下章预告

下一章学习**模拟与期权定价**：把概率分布、随机抽样与期望折现用于金融定价。

本章内容改编自教材第 11 章[1]。

[1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京 : 人民邮电出版社, 2018.



Thanks for your attention!

Q & A